

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3

з дисципліни «Бази даних»

Виконав:

Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірів:
Русінов В.В.

Київ 2026

Короткий виклад вимог

Завдання:

Використовуючи базу даних VideoHub з Lab 2, виконати базові операції CRUD (Create, Read, Update, Delete) — це операції OLTP (Online Transaction Processing), що виконуються в додатку в реальному часі.

Що таке CRUD:

- **C (Create)** — INSERT — додавання нових записів
- **R (Read)** — SELECT — читання даних з фільтрацією, сортуванням, обмеженням
- **U (Update)** — UPDATE — зміна існуючих записів
- **D (Delete)** — DELETE — видалення записів

SELECT (Read)

1. Вибір усіх даних з таблиці User:

```
SELECT * FROM "User";
```

Зірочка `*` означає «всі колонки». Повертає всі рядки та всі колонки таблиці User.

2. Вибір конкретних колонок з WHERE — відео з переглядами > 10000:

```
SELECT title, views, author_id FROM "Video" WHERE views > 10000;
```

Вибираємо тільки колонки title, views, author_id. Умова `WHERE views > 10000` фільтрує відео з більш ніж 10000 переглядами.

3. Вибір з сортуванням та LIKE:

```
SELECT username, email, created_at FROM "User" WHERE username LIKE '%a%' ORDER BY created_at;
```

`LIKE '%a%'` — пошук за шаблоном: username містить літеру 'a' (відсоток = будь-яка кількість символів до та після). `ORDER BY created_at` — сортування за датою створення (за зростанням).

4. Вибір з LIMIT — топ-3 відео за переглядами:

```
SELECT title, views FROM "Video" ORDER BY views DESC LIMIT 3;
```

ORDER BY views DESC — сортування за спаданням (найбільше переглядів першими). LIMIT 3 — тільки перші 3 рядки.

INSERT (Create)

1. Додати нового користувача:

```
INSERT INTO "User" (email, username, password, description)
VALUES ('kate@example.com', 'kate_art', 'hashed_pass_6', 'Digital artist sharing creative
process.');
```

Вказуємо тільки обов'язкові поля (email, username, password, description). Поле id генерується автоматично через DEFAULT uuid_generate_v4(), поле created_at — через DEFAULT now().

Перевірка:

```
SELECT * FROM "User" WHERE username = 'kate_art';
```

2. Додати нове відео:

```
INSERT INTO "Video" (title, description, url, thumbnail, views, author_id)
VALUES (
    'Digital Painting Tutorial – Procreate',
    'Step by step digital painting in Procreate.',
    'https://cdn.videohub.com/videos/vid6.mp4',
    'https://cdn.videohub.com/thumbs/vid6.jpg',
    0,
    (SELECT id FROM "User" WHERE username = 'kate_art')
);
```

author_id отримуємо через підзапит — знаходимо id користувача kate_art. Поле views = 0 (нове відео).

Перевірка:

```
SELECT * FROM "Video" WHERE title = 'Digital Painting Tutorial – Procreate';
```

3. Додати новий коментар:

```
INSERT INTO "Comment" (text, user_id, video_id)
VALUES (
    'Amazing tutorial, learned so much!',
    (SELECT id FROM "User" WHERE username = 'denys_dev'),
    (SELECT id FROM "Video" WHERE title = 'Digital Painting Tutorial – Procreate')
);
```

Підзапити для `user_id` та `video_id` — знаходимо ID користувача `denys_dev` та ID нового відео.

Перевірка:

```
SELECT * FROM "Comment" WHERE text = 'Amazing tutorial, learned so much!';
```

UPDATE

1. Збільшити лічильник переглядів відео на 1:

```
-- До
SELECT title, views FROM "Video" WHERE title = 'PostgreSQL Tutorial for Beginners';

-- UPDATE
UPDATE "Video" SET views = views + 1 WHERE title = 'PostgreSQL Tutorial for Beginners';

-- Після
SELECT title, views FROM "Video" WHERE title = 'PostgreSQL Tutorial for Beginners';
```

`SET views = views + 1` — збільшує поточне значення на 1. `WHERE` вказує який саме рядок оновити. Без `WHERE` — оновились би всі рядки таблиці.

2. Змінити `description` користувача:

```
-- До
SELECT username, description FROM "User" WHERE username = 'maria_gaming';

-- UPDATE
UPDATE "User" SET description = 'Gaming streamer. RPGs, indie games, and horror games.'
WHERE username = 'maria_gaming';

-- Після
SELECT username, description FROM "User" WHERE username = 'maria_gaming';
```

`SET description = '...'` — замінює значення `description`. `WHERE username = 'maria_gaming'` — фільтр для конкретного користувача.

DELETE

1. Видалити коментар (просте видалення):

```
-- До
SELECT * FROM "Comment" WHERE text = 'Amazing tutorial, learned so much!';

-- DELETE
DELETE FROM "Comment" WHERE text = 'Amazing tutorial, learned so much!';
```

```
-- Після
SELECT * FROM "Comment" WHERE text = 'Amazing tutorial, learned so much!';
```

Таблиця Comment не має дочірніх таблиць, що посилаються на неї — тому видалення проходить без помилок. WHERE вказує який рядок видалити. Без WHERE — видалились би всі рядки.

2. Видалити користувача з відео — перевірка CASCADE:

```
-- До: показуємо користувача та його відео
SELECT username FROM "User" WHERE username = 'kate_art';
SELECT title FROM "Video" WHERE author_id = (SELECT id FROM "User" WHERE username =
'kate_art');

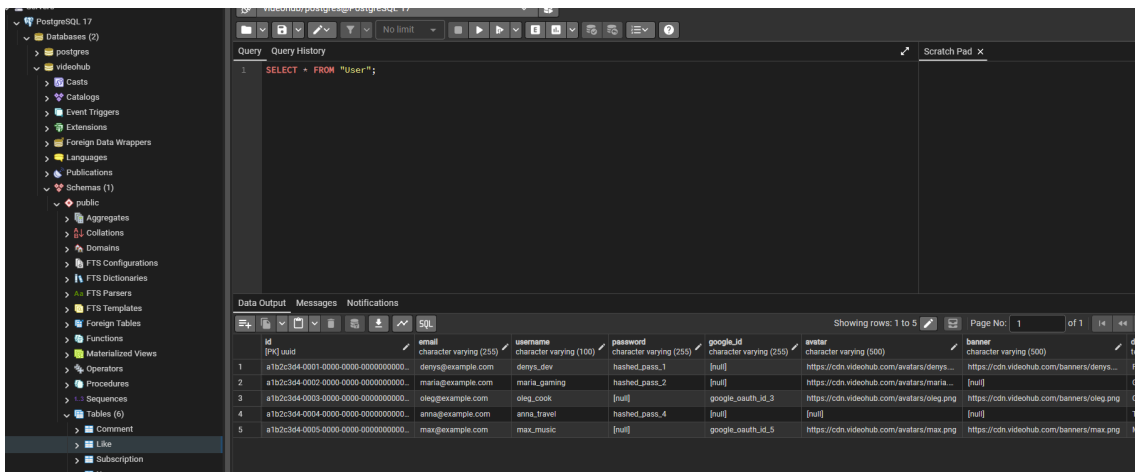
-- DELETE: видаляємо користувача → CASCADE видаляє відео
DELETE FROM "User" WHERE username = 'kate_art';

-- Після: користувача немає, відео теж немає (CASCADE)
SELECT * FROM "User" WHERE username = 'kate_art';
SELECT * FROM "Video" WHERE title = 'Digital Painting Tutorial – Procreate';
```

При видаленні користувача kate_art автоматично видаляється його відео 'Digital Painting Tutorial — Procreate' через ON DELETE CASCADE на зовнішньому ключі Video.author_id → User.id. Це демонструє каскадне видалення — основну перевагу правильно налаштованих FK.

Скріншоти

SELECT — вибір усіх користувачів:



The screenshot shows the PostgreSQL 17 interface. The left sidebar displays the database structure, including the 'public' schema and the 'User' table. The main window shows the query history with the following SQL statement:

```
SELECT * FROM "User";
```

The Data Output pane displays the results of the query, showing 5 rows of user data:

id	email	username	password	google_id	avatar	banner	description
1	a1b2c3d4-0001-0000-0000-000000000000...	denys@example.com	denys_dev	hashed_pass_1	[null]	https://cdn.videohub.com/avatars/denys...	https://cdn.videohub.com/banners/denys...
2	a1b2c3d4-0002-0000-0000-000000000000...	maria@example.com	maria_gaming	hashed_pass_2	[null]	https://cdn.videohub.com/avatars/maria...	https://cdn.videohub.com/banners/maria...
3	a1b2c3d4-0003-0000-0000-000000000000...	oleg@example.com	oleg_cook	google_auth_id_3	https://cdn.videohub.com/avatars/oleg.png	https://cdn.videohub.com/banners/oleg.png	https://cdn.videohub.com/banners/oleg.png
4	a1b2c3d4-0004-0000-0000-000000000000...	anna@example.com	anna_travel	hashed_pass_4	[null]	[null]	[null]
5	a1b2c3d4-0005-0000-0000-000000000000...	max@example.com	max_music	google_auth_id_5	https://cdn.videohub.com/avatars/max.png	https://cdn.videohub.com/banners/max.png	https://cdn.videohub.com/banners/max.png

SELECT — відео з переглядами > 10000:

The screenshot shows a PostgreSQL client interface with a sidebar on the left displaying the database structure. The main window is titled "videohub/postgres@PostgreSQL 17". The "Query" tab is active, showing the following SQL queries:

```
1 SELECT title, views, author_id FROM "Video" WHERE views > 10000;
2 SELECT username, email, created_at FROM "User" WHERE username LIKE '%a%' ORDER BY created_at;
3 SELECT title, views FROM "Video" ORDER BY views DESC LIMIT 3;
```

The "Data Output" tab is active, showing the results of the third query:

	title	views
1	Elden Ring – Final Boss Fight	89300
2	Exploring Tokyo – Shibuya at Ni...	32100
3	PostgreSQL Tutorial for Beginners	15420

INSERT — додавання нового користувача та перевірка:

The screenshot shows a PostgreSQL client interface with a sidebar on the left. The main window is titled "videohub/postgres@PostgreSQL 17". The "Query" tab is active, showing the following SQL queries:

```
1 INSERT INTO "User" (email, username, password, description)
2 VALUES ('kate@example.com', 'kate_art', 'hashed_pass_6', 'Digital artist sharing creative process.');
```

The "Data Output" tab is active, showing the results of the first query:

id	email	username	password	google_id	avatar	banner	description	created_at
1	kate@example.com	kate_art	hashed_pass_6	[null]	[null]	[null]	Digital artist sharing creative process.	2026-09-01 12:00:00

INSERT — додавання нового відео та перевірка:

videohub/postgres@PostgreSQL 17

Query Query History

```
1 INSERT INTO "Video" (title, description, url, thumbnail, views, author_id)
2 VALUES (
3     'Digital Painting Tutorial – Procreate',
4     'Step by step digital painting in Procreate.',
5     'https://cdn.videohub.com/videos/v1d6.mp4',
6     'https://cdn.videohub.com/thumbs/v1d6.jpg',
7     0,
8     (SELECT id FROM "User" WHERE username = 'kate_art')
9 );
10
11 SELECT * FROM "Video" WHERE title = 'Digital Painting Tutorial – Procreate';
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	id [PK] uuid	title character varying (255)	description text	url character varying (500)	thumbnail character varying (500)	views integer	created_at timestamp w
1	281a8dcb-214f-42e7-986c-a590edb0b...	Digital Painting Tutorial – Procre...	Step by step digital painting in Procre...	https://cdn.videohub.com/videos/vid6...	https://cdn.videohub.com/thumbs/vid6...	0	2026-09-01 2

UPDATE — збільшення переглядів (до та після):

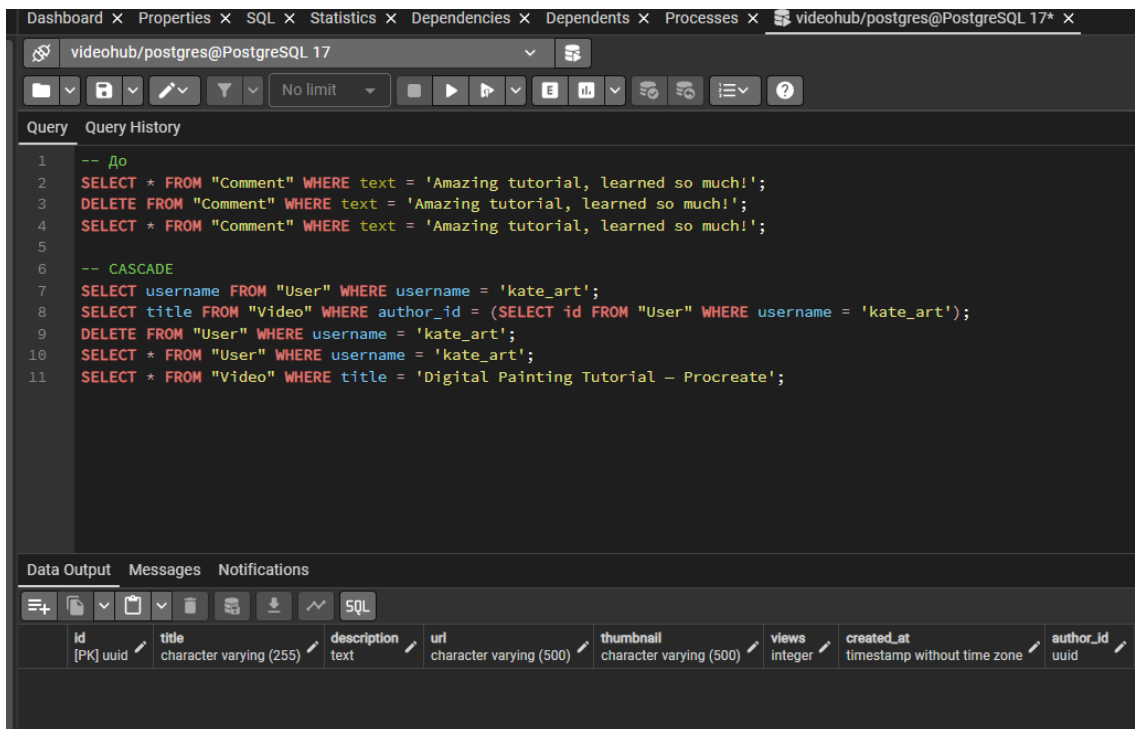
Query Query History

```
1 -- до
2 SELECT title, views FROM "Video" WHERE title = 'PostgreSQL Tutorial for Beginners';
3 -- UPDATE
4 UPDATE "Video" SET views = views + 1 WHERE title = 'PostgreSQL Tutorial for Beginners';
5 -- після
6 SELECT title, views FROM "Video" WHERE title = 'PostgreSQL Tutorial for Beginners';
```

Data Output Messages Notifications

	title character varying (255)	views integer
1	PostgreSQL Tutorial for Beginn...	15421

DELETE — видалення користувача з CASCADE:



Будь які припущення чи обмеження

- Усі операції виконуються в базі даних `videohub`, створеній у Lab 2.
- Підзапити в INSERT використовуються для отримання ID користувача та відео за відомими значеннями (username, title). У реальному додатку ID передаються напряму з коду.
- UPDATE завжди виконується з WHERE — без WHERE оновляться всі рядки таблиці, що може призвести до втрати даних.
- DELETE завжди виконується з WHERE — без WHERE видаляться всі рядки. Для повного очищення таблиці використовується TRUNCATE.
- CASCADE при DELETE User автоматично видаляє всі пов'язані записи (Video, Comment, Like, Subscription). Це запобігає «сиротам» — записам з неіснуючим FK.
- LIKE '%a%' — пошук без урахування регістру в PostgreSQL не працює за замовчуванням. Для case-insensitive пошуку використовується ILIKE.
- LIMIT 3 повертає максимум 3 рядки. Якщо менше 3 відео — поверне стільки, скільки є.